

24MCDAAT302

Design & Analysis of Algorithms

Module - 1

Algorithm Analysis

Analysis of efficiency of an algorithm can be performed at two different stages, before implementation and after implementation

- ❑ **A priori analysis** – This is defined as theoretical analysis of an algorithm. Efficiency of algorithm is measured by assuming that all other factors e.g. speed of processor, are constant and have no effect on implementation
- ❑ **A posterior analysis** – This is defined as empirical analysis of an algorithm. The chosen algorithm is implemented using programming language. Next the chosen algorithm is executed on target computer machine. In this analysis, actual statistics like running time and space needed are collected.

Algorithm Complexity

Suppose X is treated as an algorithm and N is treated as the size of input data, the time and space implemented by the Algorithm X are the two main factors which determine the efficiency of X .

- ❑ **Time Factor** – The time is calculated or measured by counting the number of key operations such as comparisons in sorting algorithm.
- ❑ **Space Factor** – The space is calculated or measured by counting the maximum memory space required by the algorithm.

The complexity of an algorithm $f(N)$ provides the running time and / or storage space needed by the algorithm with respect of N as the size of input data.

Space Complexity

Space complexity of an algorithm represents the amount of memory space needed the algorithm in its life cycle.

Space needed by an algorithm is equal to the sum of the following two components

- ❑ A **fixed part** that is a space required to store certain data and variables (i.e. simple variables and constants, program size etc.), that are not dependent of the size of the problem.
- ❑ A **variable part** is a space required by variables, whose size is totally dependent on the size of the problem. For example, recursion stack space, dynamic memory allocation etc.

Space complexity $S(p)$ of any algorithm p is $S(p) = A + S_p(I)$ Where A is treated as the fixed part and $S(I)$ is treated as the variable part of the algorithm which depends on instance characteristic I .

Algorithm

SUM(P, Q)

Step 1 - START

Step 2 - $R \leftarrow P + Q + 10$

Step 3 - Stop

Here we have three variables P, Q and R and one constant. Hence $S(p) = 1+3$. Now space is dependent on data types of given constant types and variables and it will be multiplied accordingly.

Time Complexity

Time Complexity of an algorithm is the representation of the amount of time required by the algorithm to execute to completion. Time requirements can be denoted or defined as a numerical function $t(N)$, where $t(N)$ can be measured as the number of steps, provided each step takes constant time.

For example, in case of addition of two n -bit integers, N steps are taken. Consequently, the total computational time is $t(N) = c * n$, where c is the time consumed for addition of two bits. Here, we observe that $t(N)$ grows linearly as input size increases.

There are different types of time complexities used:

1. Constant time – $O(1)$

2. Linear time – $O(n)$

3. Logarithmic time – $O(\log n)$

4. Quadratic time – $O(n^2)$

5. Cubic time – $O(n^3)$

and many more complex notations like Exponential time, Quasilinear time, factorial time, etc. are used based on the type of functions defined.

Constant time – $O(1)$

An algorithm is said to have constant time with order $O(1)$ when it is not dependent on the input size n . Irrespective of the input size n , the runtime will always be the same.

Linear time – $O(n)$

An algorithm is said to have a linear time complexity when the running time increases linearly with the length of the input. When the function involves checking all the values in input data, such function has Time complexity with this order $O(n)$.

Logarithmic time – $O(\log n)$

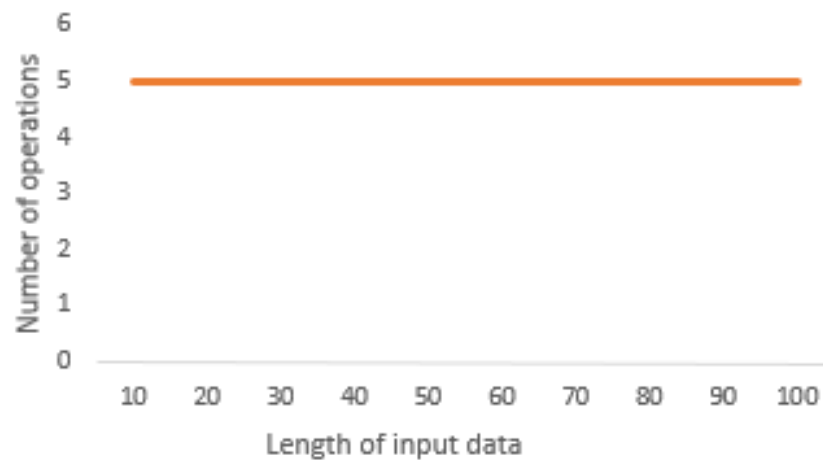
- ❑ An algorithm is said to have a logarithmic time complexity when it reduces the size of the input data in each step. This indicates that the number of operations is not the same as the input size. The number of operations gets reduced as the input size increases.
- ❑ Algorithms with Logarithmic time complexity are found in binary trees or binary search functions. This involves the search of a given value in an array by splitting the array into two and starting searching in one split. This ensures the operation is not done on every element of the data

Quadratic time – $O(n^2)$

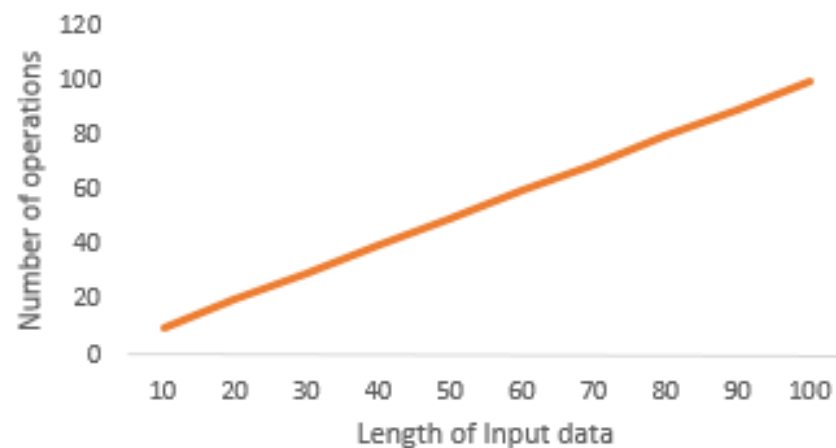
An algorithm is said to have a non – linear time complexity where the running time increases non-linearly (n^2) with the length of the input. Generally, nested loops come under this time complexity order where for one loop takes $O(n)$ and if the function involves a loop within a loop, then it goes for $O(n)*O(n) = O(n^2)$ order.

Similarly, if there are 'm' loops defined in the function, then the order is given by $O(n^m)$, which are called polynomial time complexity functions.

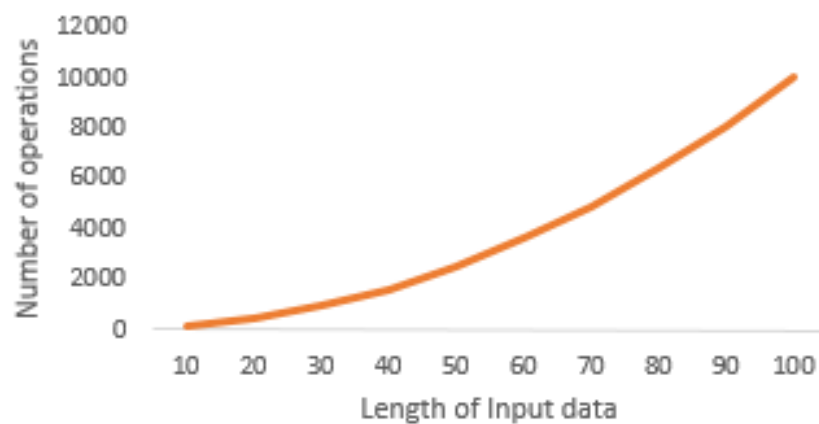
$O(1)$



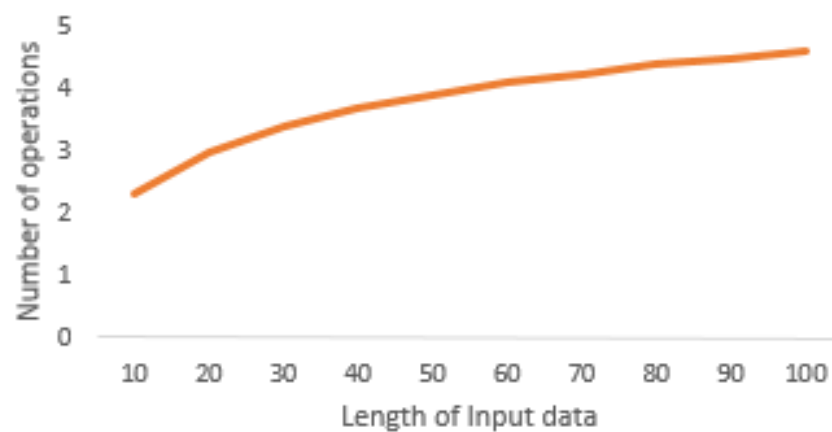
$O(n)$

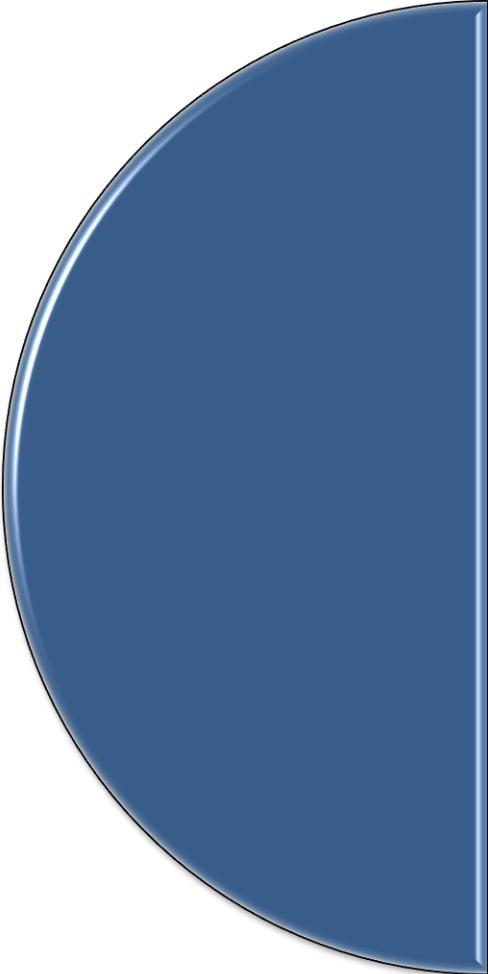


$O(n^2)$



$O(\log n)$





Asymptotic Notations,
Recurrence Equations,
Solving Recurrence
Equations- Substitution
method and Iteration
method.

Asymptotic Notation

Asymptotic Notation is used to describe the running time of an algorithm - how much time an algorithm takes with a given input, n . There are three different notations: big O, big Theta (Θ), and big Omega (Ω). big- Θ is used when the running time is the same for all cases, big-O for the worst case running time, and big- Ω for the best case running time.

Asymptotic analysis refers to computing the running time of any operation in mathematical units of computation. For example, the running time of one operation is computed as $f(n)$ and may be for another operation it is computed as $g(n^2)$. This means the first operation running time will increase linearly with the increase in n and the running time of the second operation will increase exponentially when n increases. Similarly, the running time of both operations will be nearly the same if n is significantly small.

Time required by an algorithm falls under three types

- ☐ **Best Case – Minimum time required for program execution.**
- ☐ **Average Case – Average time required for program execution.**
- ☐ **Worst Case – Maximum time required for program execution.**

Big-O Notation

The Big-O notation describes the worst-case running time of a program. We compute the Big-O of an algorithm by counting how many iterations an algorithm will take in the worst-case scenario with an input of N . We typically consult the Big-O because we must always plan for the worst case.

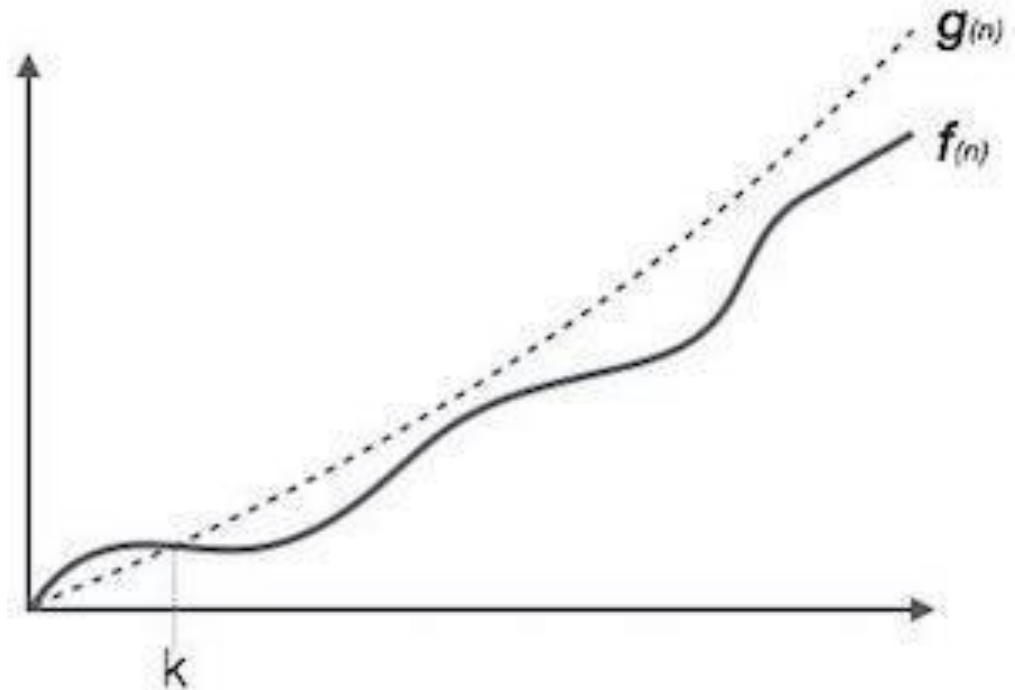
For example, $O(\log n)$ describes the Big-O of a binary search algorithm.

Big Oh Notation, O

The notation $O(n)$ is the formal way to express the upper bound of an algorithm's running time. It measures the worst case time complexity or the longest amount of time an algorithm can possibly take to complete.

For example, for a function $f(n)$

$O(f(n)) = \{ g(n) : \text{there exists } c > 0 \text{ and } n_0 \text{ such that } f(n) \leq c \cdot g(n) \text{ for all } n > n_0. \}$



Big- Ω Notation

Big- Ω (Omega) describes the best running time of a program. We compute the big- Ω by counting how many iterations an algorithm will take in the best-case scenario based on an input of N .

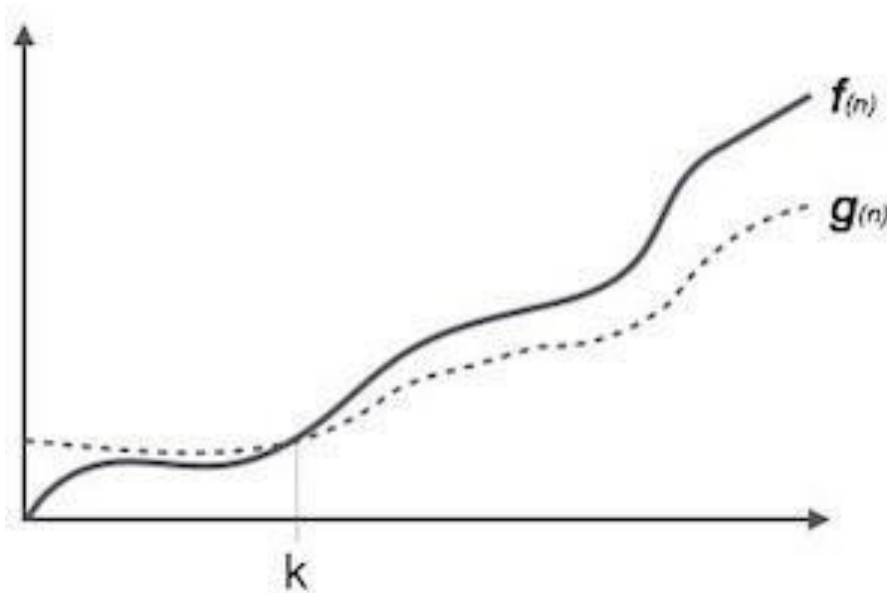
For example, a Bubble Sort algorithm has a running time of $\Omega(N)$ because in the best case scenario the list is already sorted, and the bubble sort will terminate after the first iteration.

Omega Notation, Ω

The notation $\Omega(n)$ is the formal way to express the lower bound of an algorithm's running time. It measures the best case time complexity or the best amount of time an algorithm can possibly take to complete.

For example, for a function $f(n)$

$\Omega(f(n)) \geq \{ g(n) : \text{there exists } c > 0 \text{ and } n_0 \text{ such that } g(n) \leq c \cdot f(n) \text{ for all } n > n_0. \}$



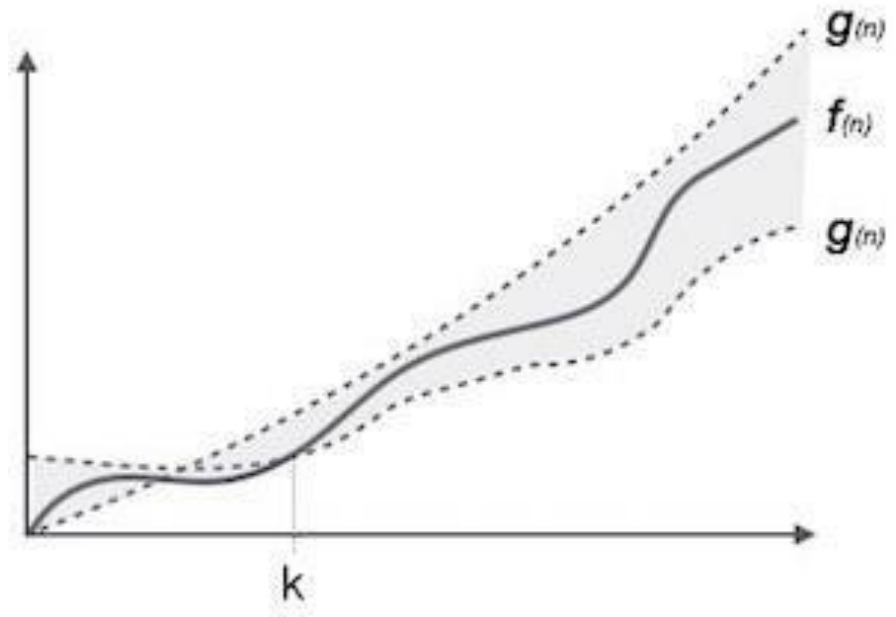
Theta Notation, θ

The notation $\theta(n)$ is the formal way to express both the lower bound and the upper bound of an algorithm's running time. It is represented as follows –

$\theta(f(n)) = \{ g(n) \text{ if and only if}$

$g(n) = O(f(n)) \text{ and } g(n) =$

$\Omega(f(n)) \text{ for all } n > n_0. \}$



Queue Versus Stack

A Queue data structure is based on First In First Out order. It takes $O(1)$ runtime to retrieve the first item in a Queue. A Stack data structure is based on First In Last Out order. Therefore, it takes $O(N)$ runtime to retrieve the first value in a Stack because it is all the way at the bottom.

Recurrence Equation

A recurrence is an equation or inequality that describes a function in terms of its values on smaller inputs. To solve a Recurrence Relation means to obtain a function defined on the natural numbers that satisfy the recurrence.

A recurrence is an equation or inequality that reflects the value of a function with smaller inputs. A recurrence can be used to represent the running duration of an algorithm that comprises a recursive call to itself. Time complexities are readily approximated by recurrence relations in many algorithms, specifically divide and conquer algorithms.

1) Substitution Method:

- ❑ Guess the Solution.
- ❑ Use the mathematical induction to find the boundary condition and shows that the guess is correct.

Solve the equation by Substitution Method.

$$T(n) = T\left(\frac{n}{2}\right) + n$$

We have to show that it is asymptotically bound by $O(\log n)$.

Solution:

For $T(n) = O(\log n)$

We have to show that for some constant c

$$T(n) \leq c \log n.$$

Put this in given Recurrence Equation.

$$\begin{aligned}T(n) &\leq c \log\left(\frac{n}{2}\right) + 1 \\&\leq c \log\left(\frac{n}{2}\right) + 1 = c \log n - c \log_2 2 + 1 \\&\leq c \log n \text{ for } c \geq 1\end{aligned}$$

Thus **$T(n) = O \log n$** .

2) Iteration Methods

It means to expand the recurrence and express it as a summation of terms of n and initial condition.

Example1: Consider the Recurrence

$$\begin{aligned} T(n) &= 1 \text{ if } n=1 \\ &= 2T(n-1) \text{ if } n>1 \end{aligned}$$

Solution:

$$\begin{aligned} T(n) &= 2T(n-1) \\ &= 2[2T(n-2)] = 2^2T(n-2) \\ &= 4[2T(n-3)] = 2^3T(n-3) \\ &= 8[2T(n-4)] = 2^4T(n-4) \text{ (Eq.1)} \end{aligned}$$

Repeat the procedure for i times

$$T(n) = 2^i T(n-i)$$

Put $n-i=1$ or $i= n-1$ in (Eq.1)

$$\begin{aligned} T(n) &= 2^{n-1} T(1) \\ &= 2^{n-1} \cdot 1 \{T(1) = 1 \text{given}\} \\ &= 2^{n-1} \end{aligned}$$

3) Recurrence Tree Method:

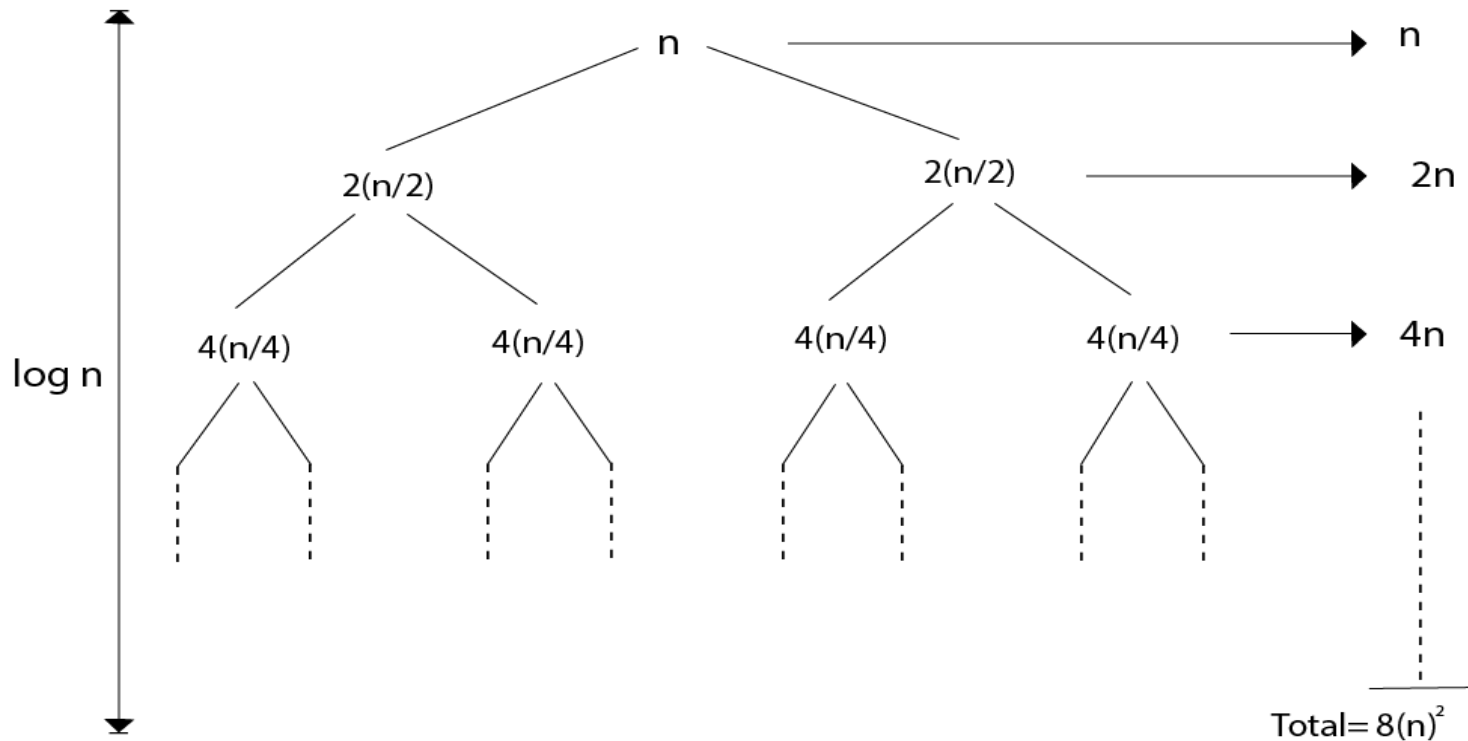
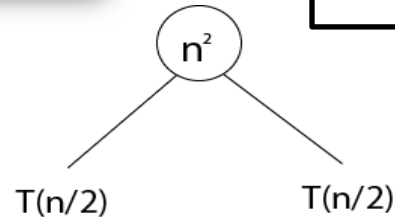
- ❑ Recursion Tree Method is a pictorial representation of an iteration method which is in the form of a tree where at each level nodes are expanded.
- ❑ In general, we consider the second term in recurrence as root.
- ❑ It is useful when the divide & Conquer algorithm is used.
- ❑ It is sometimes difficult to come up with a good guess. In Recursion tree, each root and child represents the cost of a single sub problem.
- ❑ We sum the costs within each of the levels of the tree to obtain a set of pre-level costs and then sum all pre-level costs to determine the total cost of all levels of the recursion.
- ❑ A Recursion Tree is best used to generate a good guess, which can be verified by the Substitution Method.

Consider the following recurrence

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Obtain the asymptotic bound using recursion tree method.

Solution: The recursion trees for the above recurrence



We have $n + 2n + 4n + \dots \log_2 n$ times

$$= n (1 + 2 + 4 + \dots \log_2 n \text{ times})$$

$$= n \frac{(2^{\log_2 n} - 1)}{(2 - 1)} = \frac{n(n - 1)}{1} = n^2 - n = \theta(n^2)$$

$$\mathbf{T(n) = \theta(n^2)}$$

4) Master Method

The Master Method is used for solving the following types of recurrence

$T(n) = a T\left(\frac{n}{b}\right) + f(n)$ with $a \geq 1$ and $b \geq 1$ be constant & $f(n)$ be a function and $\frac{n}{b}$ can be interpreted as

Let $T(n)$ is defined on non-negative integers by the recurrence.

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

In the function to the analysis of a recursive algorithm, the constants and function take on the following significance:

- ❑ n is the size of the problem.
- ❑ a is the number of sub problems in the recursion.
- ❑ n/b is the size of each sub problem. (Here it is assumed that all sub problems are essentially the same size.)
- ❑ $f(n)$ is the sum of the work done outside the recursive calls, which includes the sum of dividing the problem and the sum of combining the solutions to the sub problems.
- ❑ It is not possible always bound the function according to the requirement, so we make three cases which will tell us what kind of bound we can apply on the function.

Divide and Conquer

Divide and Conquer

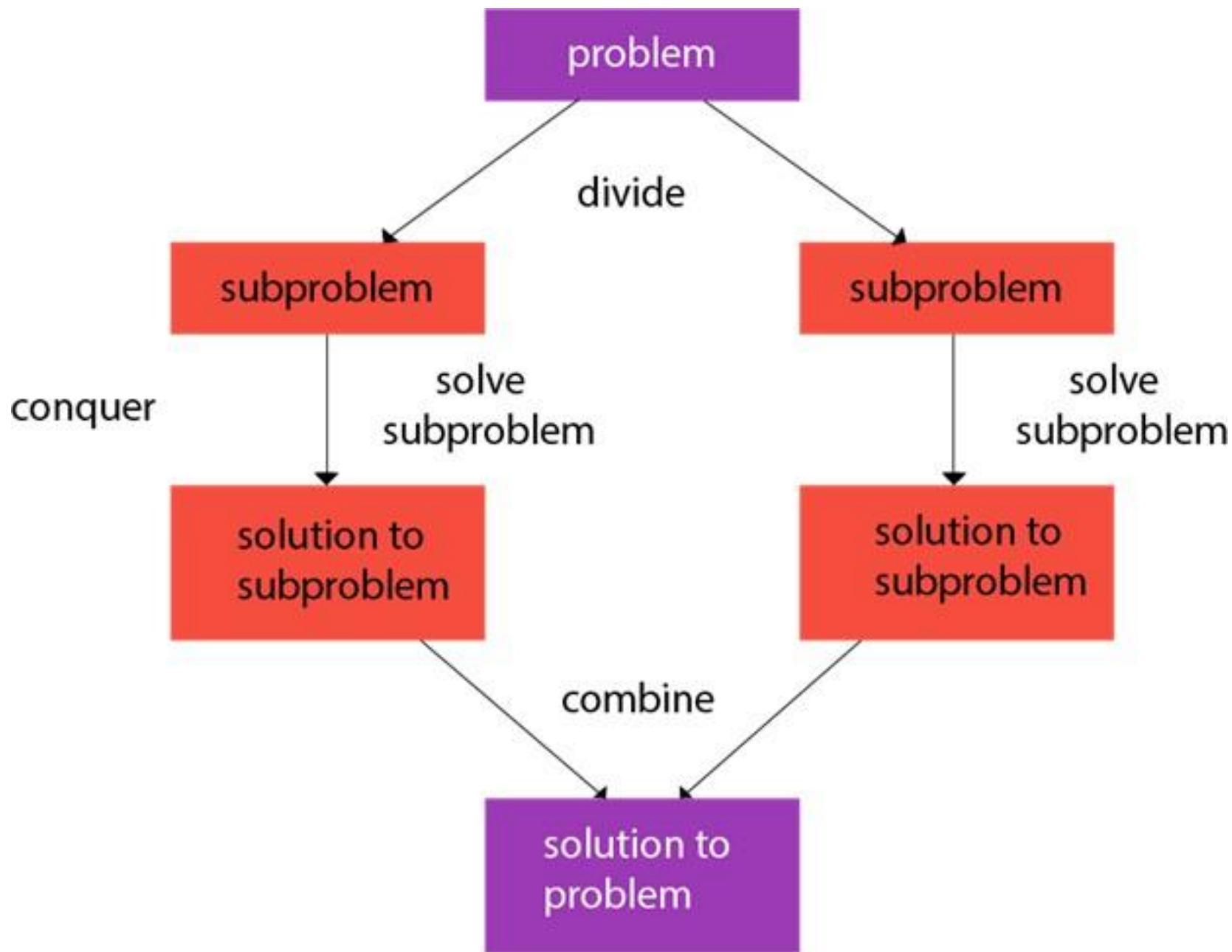
Divide and Conquer is an algorithmic pattern. In algorithmic methods, the design is to take a dispute on a huge input, break the input into minor pieces, decide the problem on each of the small pieces, and then merge the piecewise solutions into a global solution. This mechanism of solving the problem is called the Divide & Conquer Strategy.

Divide and Conquer algorithm consists of a dispute using the following three steps.

Divide the original problem into a set of sub problems.

Conquer: Solve every sub problem individually, recursively.

Combine: Put together the solutions of the sub problems to get the solution to the whole problem.



Advantages of Divide and Conquer

- ❑ Divide and Conquer tend to successfully solve one of the biggest problems, such as the Tower of Hanoi, a mathematical puzzle. It is challenging to solve complicated problems for which you have no basic idea, but with the help of the divide and conquer approach, it has lessened the effort as it works on dividing the main problem into two halves and then solve them recursively. This algorithm is much faster than other algorithms.
- ❑ It efficiently uses cache memory without occupying much space because it solves simple sub problems within the cache memory instead of accessing the slower main memory.
- ❑ It is more proficient than that of its counterpart Brute Force technique.
- ❑ Since these algorithms inhibit parallelism, it does not involve any modification and is handled by systems incorporating parallel processing.

Disadvantages of Divide and Conquer

- ❑ Since most of its algorithms are designed by incorporating recursion, so it necessitates high memory management.
- ❑ An explicit stack may overuse the space.
- ❑ It may even crash the system if the recursion is performed rigorously greater than the stack present in the CPU.

Control Abstraction of Divide and Conquer

A control abstraction is a procedure whose flow of control is clear but whose primary operations are specified by other procedures whose precise meanings are left undefined.

The control abstraction for divide and conquer technique is DANDC(P), where P is the problem to be solved.

DANDC (P)

```
{  
    if SMALL (P) then return S (p);  
    else  
    {  
        divide p into smaller instances  $p_1, p_2, \dots, p_k, k \geq 1$ ;  
        apply DANDC to each of these sub problems;  
        return (COMBINE (DANDC ( $p_1$ ) , DANDC ( $p_2$ ),..., DANDC ( $p_k$ )));  
    }  
}
```

SMALL (P) is a Boolean valued function which determines whether the input size is small enough so that the answer can be computed without splitting. If this is so function 'S' is invoked otherwise, the problem 'p' into smaller sub problems. These sub problems $p_1, p_2, \dots, p_k$ are solved by recursive application of DANDC. If the sizes of the two sub problems are approximately equal then the computing time of DANDC is:

$$T(n) = \begin{cases} g(n) & n \text{ small} \\ 2T(n/2) + f(n) & \text{otherwise} \end{cases}$$

Where, $T(n)$ is the time for DANDC on 'n' inputs
 $g(n)$ is the time to complete the answer directly for small inputs and
 $f(n)$ is the time for Divide and Combine

Merge Sort

This sorting algorithm that falls under the category of Divide and Conquer technique. It is one of the best sorting techniques that successfully build a recursive algorithm.

Divide and Conquer Strategy

In this technique, we segment a problem into two halves and solve them individually. After finding the solution of each half, we merge them back to represent the solution of the main problem.

Suppose we have an array A , such that our main concern will be to sort the subsection, which starts at index p and ends at index r , represented by $A[p..r]$.

Divide

If assumed q to be the central point somewhere in between p and r , then we will fragment the subarray $A[p..r]$ into two arrays $A[p..q]$ and $A[q+1, r]$.

Conquer

After splitting the arrays into two halves, the next step is to conquer. In this step, we individually sort both of the subarrays $A[p..q]$ and $A[q+1, r]$. In case if we did not reach the base situation, then we again follow the same procedure, i.e., we further segment these subarrays followed by sorting them separately.

Combine

As when the base step is acquired by the conquer step, we successfully get our sorted subarrays $A[p..q]$ and $A[q+1, r]$, after which we merge them back to form a new sorted array $[p..r]$.

Merge() Function

Consider the following example of an unsorted array, which we are going to sort with the help of the Merge Sort algorithm.

A= (36,25,40,2,7,80,15)

Step1: The merge sort algorithm iteratively divides an array into equal halves until we achieve an atomic value. In case if there are an odd number of elements in an array, then one of the halves will have more elements than the other half.

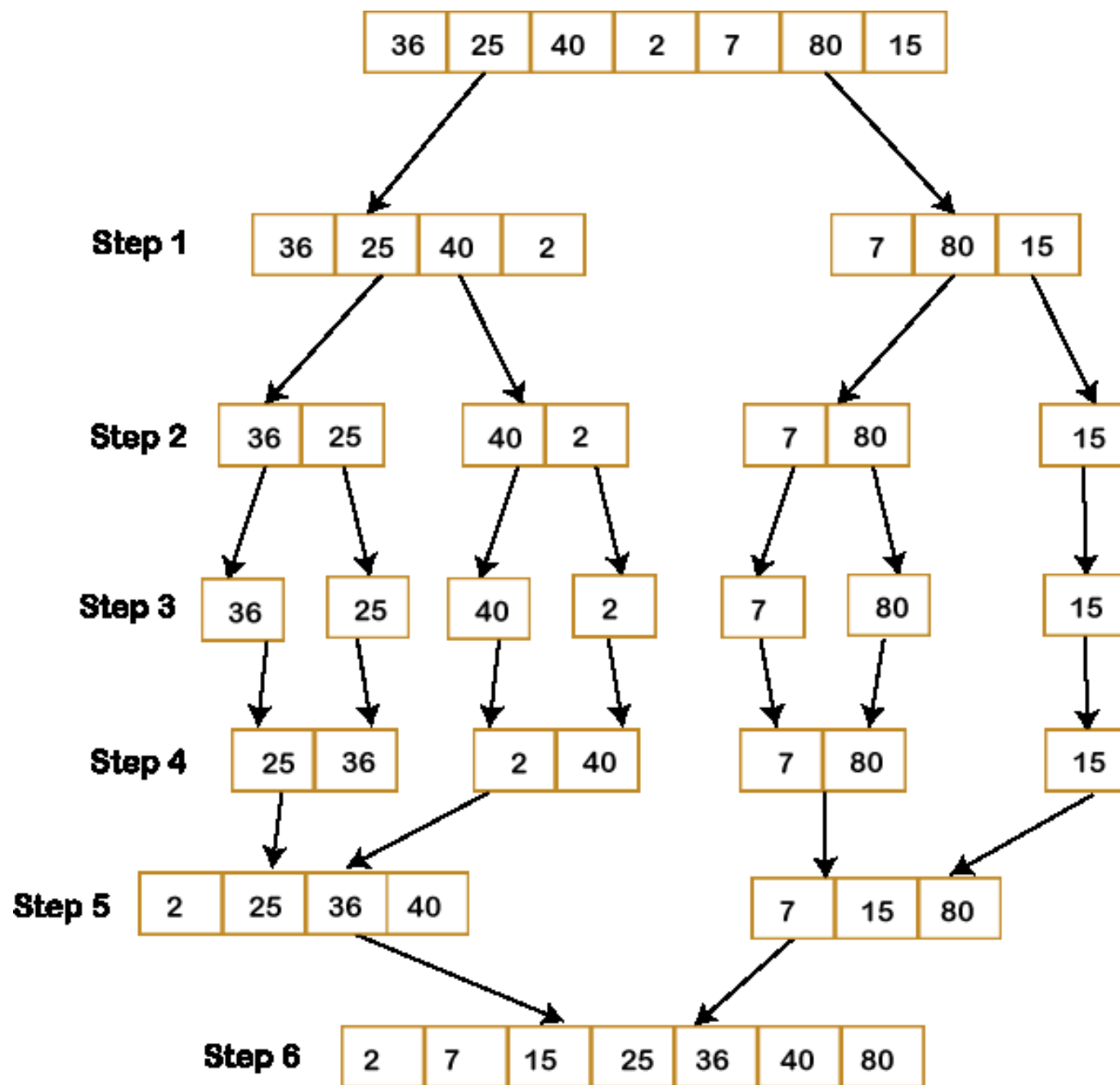
Step2: After dividing an array into two subarrays, we will notice that it did not hamper the order of elements as they were in the original array. After now, we will further divide these two arrays into other halves.

Step3: Again, we will divide these arrays until we achieve an atomic value, i.e., a value that cannot be further divided.

Step4: Next, we will merge them back in the same way as they were broken down.

Step5: For each list, we will first compare the element and then combine them to form a new sorted list.

Step6: In the next iteration, we will compare the lists of two data values and merge them back into a list of found data values, all placed in a sorted manner.



Analysis of Merge Sort:

Let $T(n)$ be the total time taken by the Merge Sort algorithm.

- Sorting two halves will take at the most $2T \frac{n}{2}$ time.
- When we merge the sorted lists, we come up with a total $n-1$ comparison because the last element which is left will need to be copied down in the combined list, and there will be no comparison.

Thus, the relational formula will be

$$T(n) = 2T\left(\frac{n}{2}\right) + n - 1$$

But we ignore '-1' because the element will take some time to be copied in merge lists.

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + in$$

$$= 2^i \times 0 + \log_2 n \cdot n$$

$$= T(n) = n \cdot \log n$$

❑ **Best Case Complexity:** The merge sort algorithm has a best-case time complexity of $O(n \cdot \log n)$ for the already sorted array.

❑ **Average Case Complexity:** The average-case time complexity for the merge sort algorithm is $O(n \cdot \log n)$, which happens when 2 or more elements are jumbled, i.e., neither in the ascending order nor in the descending order.

❑ **Worst Case Complexity:** The worst-case time complexity is also $O(n \cdot \log n)$, which occurs when we sort the descending order of an array into the ascending order.

❑ **Space Complexity:** The space complexity of merge sort is $O(n)$.

Quick sort

It is an algorithm of Divide & Conquer type.

❑ Divide: Rearrange the elements and split arrays into two sub-arrays and an element in between such that each element in left sub array is less than or equal to the average element and each element in the right sub- array is larger than the middle element.

❑ Conquer: Recursively, sort two sub arrays.

❑ Combine: Combine the already sorted array.

Example of Quick Sort:

44 33 11 55 77 90 40 60 99 22 88

Let 44 be the Pivot element and scanning done from right to left

Comparing 44 to the right-side elements, and if right-side elements are smaller than 44, then swap it. As 22 is smaller than 44 so swap them.

22	33	11	55	77	90	40	60	99	44	88
----	----	----	----	----	----	----	----	----	----	----

Now comparing 44 to the left side element and the element must be **greater** than 44 then swap them. As 55 are greater than 44 so swap them.

22	33	11	44	77	90	40	60	99	55	88
----	----	----	----	----	----	----	----	----	----	----

Recursively, repeating steps 1 & steps 2 until we get two lists one left from pivot element 44 & one right from pivot element.

22	33	11	40	77	90	44	60	99	55	88
----	----	----	----	----	----	----	----	----	----	----

Swap with 77:

22	33	11	40	44	90	77	60	99	55	88
----	----	----	----	----	----	----	----	----	----	----

Now, the element on the right side and left side are greater than and smaller than 44 respectively.

Now we get two sorted lists:

22	33	11	40	44	90	77	66	99	55	88
Sublist1					Sublist2					

And these sublists are sorted under the same process as above done.

These two sorted sublists side by side.

22	33	11	40	44	90	77	60	99	55	88
11	33	22	40	44	88	77	60	99	55	90
11	22	33	40	44	88	77	60	90	55	99

First sorted list

88	77	60	55	90	99
----	----	----	-----------	-----------	----

Sublist3

Sublist4

55	77	60	88	90	99
----	----	----	-----------	----	----

Sorted

55	77	60
----	-----------	-----------

55	60	77
----	----	----

Sorted

Merging Sublists:

11	22	33	40	44	55	60	77	88	90	99
----	----	----	----	----	----	----	----	----	----	----

Worst Case Analysis: It is the case when items are already in sorted form and we try to sort them again. This will takes lots of time and space.

Equation:

$$T(n) = T(1) + T(n-1) + n$$

$T(1)$ is time taken by pivot element.

$T(n-1)$ is time taken by remaining element except for pivot element.

N : the number of comparisons required to identify the exact position of itself (every element)

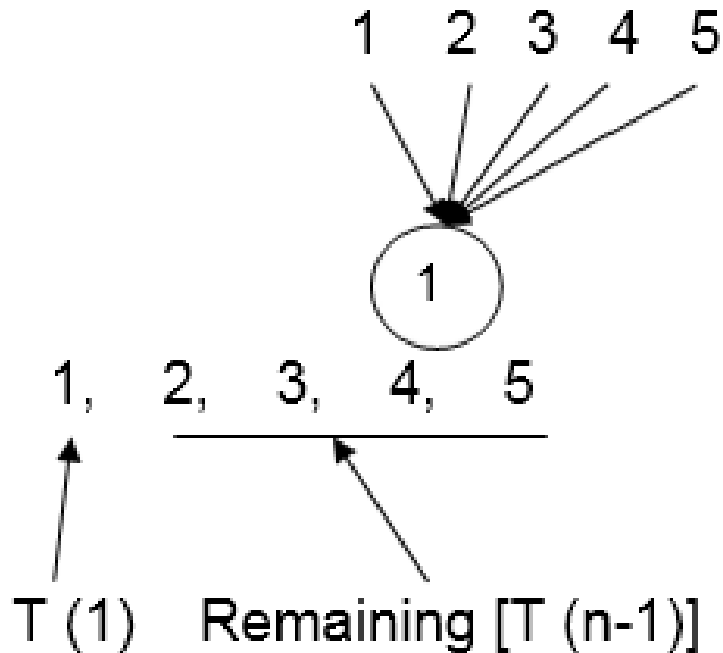
Quick sort follows the below steps:

Step 1 – Make any element as pivot

Step 2 – Partition the array on the basis of pivot

Step 3 – Apply quick sort on left partition recursively

Step 4 – Apply quick sort on right partition recursively

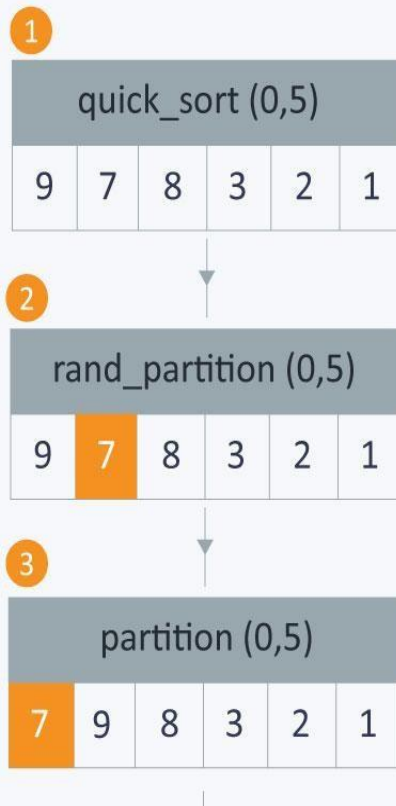


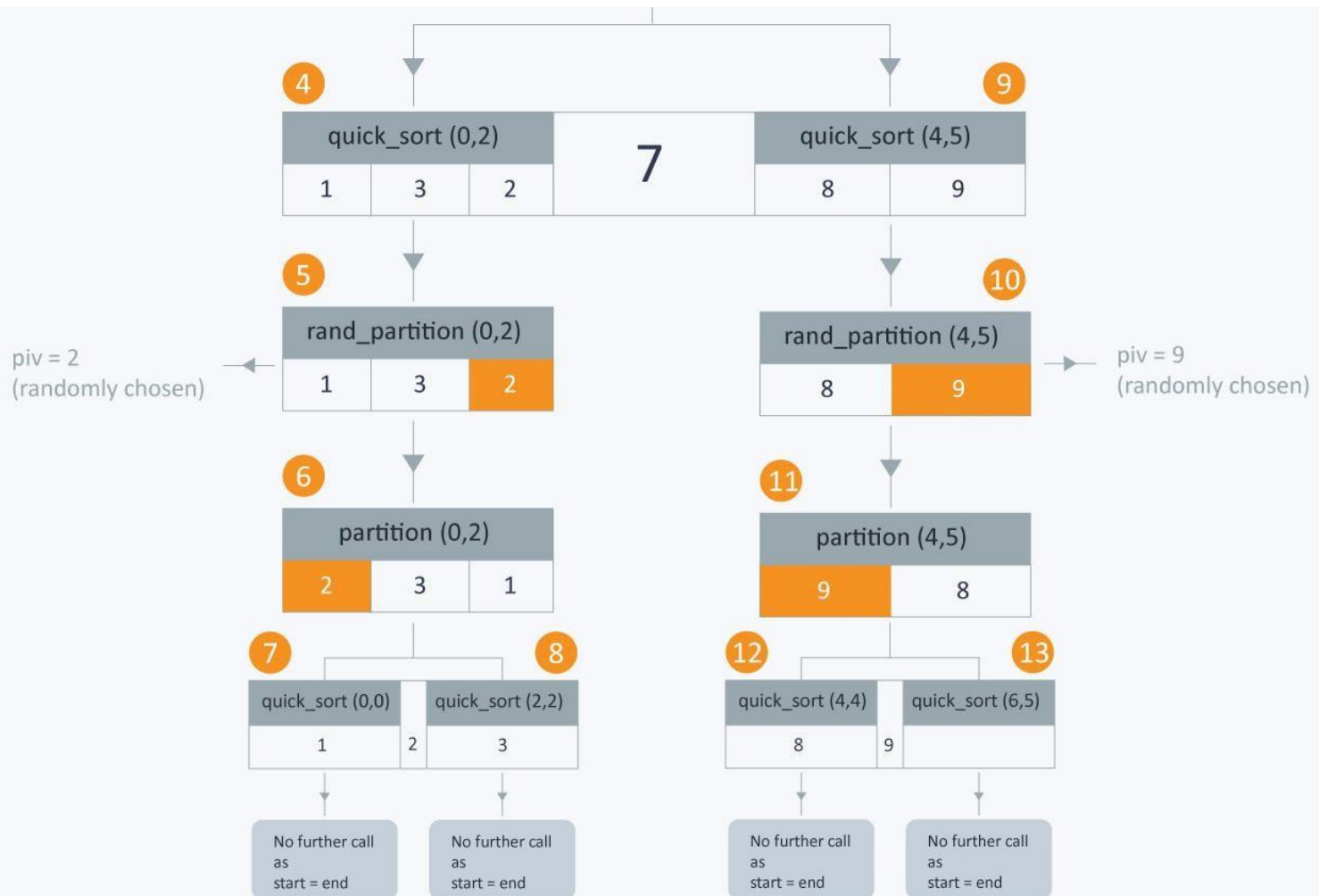
Worst Case Complexity of Quick Sort is $T(n) = O(n^2)$

Method Name	Equation	Stopping Condition	Complexities
1.Quick Sort[Worst Case]	$T(n)=T(n-1)+T(0)+n$	$T(1)=0$	$T(n)=n^2$
2.Quick Sort[Average Case]	$T(n)=n+1 + \frac{1}{n}$ $(\sum_{k=1}^n T(k-1) + T(n-k))$		$T(n)=n \log n$

Quick Sort [Best Case]: In any sorting, best case is the only case in which we don't make any comparison between elements that is only done when we have only one element to sort.

Quick Sort





Example: You have an array $A=[9,7,8,3,2,1]$ Observe in the diagram below, that the `randpartition()` function chooses pivot randomly as 7 and then swaps it with the first element of the array and then the `partition()` function call takes place, which divides the array into two halves. The first half has elements less than 7 and the other half has elements greater than 7.

For elements less than 7, in 5th call, `randpartition()` function chooses 2 as pivot element randomly and then swap it with first element and call to the `partition()` function takes place. After the 7th and 8th call, no further calls can take place as only one element left in both the calls. Similarly, you can observe the order of calls for the elements greater than 7.

Use randpartition instead of partition function in quicksort function to reduce the time complexity of this algorithm.

Complexity The worst case time complexity of this algorithm is $O(n^2)$, but as this is randomized algorithm, its time complexity fluctuates between $O(n \log n)$ and $O(n^2)$ and mostly it comes out to be $O(n \log n)$.

Matrix Multiplication

To solve our problem assume 2×2 is the smallest square matrix. Let A and B are two different matrices.

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$$

$$B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

$$c_{11} = a_{11} * b_{11} + a_{12} * b_{21}$$

$$c_{12} = a_{11} * b_{12} + a_{12} * b_{22}$$

$$c_{21} = a_{21} * b_{11} + a_{22} * b_{21}$$

$$c_{22} = a_{21} * b_{12} + a_{22} * b_{22}$$

$$C = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$$

Where $C = A * B$, The Matrix C can be calculated as,

$$c_{11} = a_{11} * b_{11} + a_{12} * b_{21}$$

$$c_{12} = a_{11} * b_{12} + a_{12} * b_{22}$$

$$c_{21} = a_{21} * b_{11} + a_{22} * b_{21}$$

$$c_{22} = a_{21} * b_{12} + a_{22} * b_{22}$$

Since this method requires 8 multiplication and 4 addition, therefore, it requires constant time.

What if the size is greater than 2×2 ?

We assume that the matrices are having dimensions in powers of 2 like 2×2 , 4×4 , 8×8 , 16×16 , 256×256 , and e.t.c. If it is not of power 2×2 then we can fill zeros and makes it a square matrix of power of 2×2 .

Example Using 4×4

Since it is a larger problem therefore we must divide it into smaller problems and then solve those sub-problem and combine the solutions.

Following is the simple divide and conquer method to multiply two 4×4 square matrices,

- a) Divide both matrices in 4 sub-matrices of size 2×2 (i.e. $n/2 \times n/2$ where $n=4$)
- b) Then calculate the values recursively.

$$A = \left[\begin{array}{cc|cc} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ \hline a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{array} \right]$$

Diagram illustrating the structure of matrix A . The matrix is partitioned into four quadrants. The top-left quadrant is labeled A_{11} with an upward arrow. The top-right quadrant is labeled A_{12} with an upward arrow. The bottom-left quadrant is labeled A_{21} with a downward arrow. The bottom-right quadrant is labeled A_{22} with a downward arrow.

$$B = \left[\begin{array}{cc|cc} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ \hline b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{array} \right]$$

Diagram illustrating the structure of matrix B . The matrix is partitioned into four quadrants. The top-left quadrant is labeled B_{11} with an upward arrow. The top-right quadrant is labeled B_{12} with an upward arrow. The bottom-left quadrant is labeled B_{21} with a downward arrow. The bottom-right quadrant is labeled B_{22} with a downward arrow.

Algorithm of Divide and Conquer for Matrix Multiplication

MatrixMultiply(a, b)

n = a.rows

let c be a new n x n matrix

if n == 1

 c11 = a11 * b11

else partition a, b, and c

 C11 = MatrixMultiply(A11, B11) + MatrixMultiply(A12, B21)

 C12 = MatrixMultiply(A11, B12) + MatrixMultiply(A12, B22)

 C21 = MatrixMultiply(A21, B11) + MatrixMultiply(A22, B21)

 C22 = MatrixMultiply(A21, B12) + MatrixMultiply(A22, B22)

return c

It has 8 recursive function calls which call itself, and it contains 4 addition. These additions are matrices addition, not the normal addition. The time complexity for the addition of two matrices is $O(N^2)$.

Recurrence relation $T(N)$,

$\Rightarrow 8T(N/2) + O(N^2)$ if $n > 1$

$\Rightarrow O(1)$ if $n = 1$

Time complexity = $O(N^3)$

Whether we are using the naive method or the divide-conquer method to find matrix multiplication its time complexity is $O(N^3)$.

The divide and conquer method uses the recursion technique, therefore, it internally uses the stack and consumes extra spaces. In terms of space complexity, the basic naive method is better than the divide and conquer technique of matrix multiplication.